

Módulo 9: Evaluación

Text Mining en R

El archivo ***evaluacion.RData*** contiene tres datasets:

- ***prado***: Cuenta de X (antiguo Twitter) del *Museo del Prado*: 1.038 tuits (desde 01-04-2025 al 31-07-2026).
- ***reviews***: Contiene 13.000 críticas sobre el alojamiento en diversas ciudades de Andalucía.
- ***documentos***: Contiene 120 oraciones.

Pregunta 1. Text Mining

El dataset ***prado*** contiene 1.038 tuits, de la cuenta del *Museo del Prado*. Descripción de las columnas:

- ***status***: Identificador del tuit
- ***fecha***: Fecha de publicación del tuit. Desde el 01-04-2025 hasta el 31-07-2026.
- ***texto***: El texto del tuit.

Para realizar *text mining* al texto de los tuits, es importante **depurar previamente el texto**.

Como, por ejemplo, eliminar del texto: usuarios de X, hashtags, URLs, ...

Para realizar la *limpieza del texto*, podemos utilizar el siguiente script:

```
# === Limpieza de Texto ===
prado$textmining <- prado$texto
# Quitar "@usuario"
prado$textmining <- gsub("@\\w+", "", prado$textmining)
# Quitar "#hashtag"
prado$textmining <- gsub("#\\w+", "", prado$textmining)
# Quitar "URL"
prado$textmining <- gsub("http[[:space:]]*", "", prado$textmining)
# Quitar "e-Mail"
prado$textmining <- gsub("\\w+@\\w+\\.\\w+", "", prado$textmining)
# Si fuera necesario, añadir espacio después de "punto", "coma", "punto y coma", "dos puntos"
prado$textmining <- gsub("([.],|;|:)([[:alpha:]])", "\\1 \\2", prado$textmining)
# Conservar letras, dígitos, espacios en blanco, saltos de línea y caracteres de puntuación
prado$textmining <- gsub("[^[:alpha:][:digit:][:space:][:punct:]]*", "", prado$textmining)
# Quitar símbolos
prado$textmining <- gsub("■", "", prado$textmining)
prado$textmining <- gsub("i", "", prado$textmining)
prado$textmining <- gsub("→", "", prado$textmining)
prado$textmining <- gsub("△", "", prado$textmining)
# Quitar espacios en blanco al inicio y final del texto
prado$textmining <- trimws(prado$textmining)
```

Ahora que el texto está *limpio*, podríamos realizar un *análisis lingüístico* y *text mining*.

Para realizar el *análisis lingüístico* sobre `prado$textmining`, es importante conservar el carácter *punto*, el cual permite crear las oraciones.

[[:alpha:]] Letras mayúsculas y minúsculas.

[[:digit:]] Dígitos.

[[:blank:]] Espacio y tabulador.

[[:space:]] Espacio, tabulador y salto de línea.

[[:punct:]] Caracteres de puntuación: ! # \$ % & ' () * + , - . / : ; < = > ? @ [] ^ _ ` { | } ~

Realizar *text mining* a los tuits de la cuenta *prado*, desde 01-06-2025 hasta 31-07-2026.

- Análisis lingüístico: trabajar con la columna *textmining*.
- Términos más frecuentes.
- Buscar *palabras clave*. Seleccionar sustantivos, adjetivos y nombres propios:
 - RAKE (Rapid Automatic Keyword Extraction).
 - PMI (Pointwise Mutual Information).
 - Extracción de Frases (frases nominales).
 - TextRank.
- Visualizar las *coocurrencias*. Seleccionar sustantivos, adjetivos y nombres propios.
- Visualizar las *correlaciones*. Seleccionar sustantivos, adjetivos y nombres propios.
- Podemos clasificar los tuits conforme a una temática determinada. Para extraer temas, podemos basarnos en la información que se desprende del *text mining* (fundamentalmente de las coocurrencias y correlaciones). Se pide:
 - Examinando las coocurrencias/correlaciones obtenidas, indica algunos nombres de *pintores* y *exposiciones*. Si te resulta más cómodo, señala las relaciones oportunas (término1 – término2 – término3 – término4).
 - Selecciona aquellos tuits donde figure la cadena de texto "Veronese". Después, muestra el texto de los cinco primeros tuits (ordenados por fecha, del tuit más antiguo al más reciente). Para finalizar, visualiza las *coocurrencias*.
 - Ídem a la pregunta anterior, pero empleando la *cadena de texto* "femenino".
- *Word embeddings*:
 - Modelo *GloVe*. El texto debe estar compuesto únicamente por los lemas que sean sustantivos, adjetivos o nombres propios.
 - Mostrar las *similitudes* de los términos: "farnesio", "caravaggio", "università", "falomir".
 - Visualizar word embeddings mediante *UMAP*.

Pregunta 2. Clasificación de Texto: Análisis de sentimiento

El dataset *reviews* contiene 13.000 críticas sobre el alojamiento en diversas ciudades de Andalucía. Descripción de las columnas:

- *titulo*: Breve descripción de la crítica.
- *texto*: El texto de la crítica.
- *puntuacion*: La calificación del usuario, en una escala de 5 estrellas. Variable categórica, con cinco valores posibles: 1 (muy mal), 2 (mal), 3 (regular), 4 (bien), 5 (muy bien).

Se pide:

- Divida *reviews* en dos datasets: dataset de *entrenamiento* (con el 80% de las filas) y dataset de *prueba* (con el 20% de las filas restantes). No es necesario que los datasets estén balanceados.
- Mediante la herramienta *fastText*, cree un *modelo de clasificación de texto* basándose en la variable *puntuacion*.
- En *reviews*, cree una variable categórica, denominada *sentimiento*, siguiendo estas indicaciones:
 - Si *puntuacion* es menor de 3, tomará el valor *negativo*.
 - Si *puntuacion* es igual a 3, tomará el valor *neutro*.
 - Si *puntuacion* es mayor de 3, tomará el valor *positivo*.Empleando la herramienta *fastText*, cree un *modelo de clasificación de texto* basándose en la variable *sentimiento*.
- Utilizando la herramienta *fastText*, cree un *modelo de clasificación de texto* basándose en la variable *sentimiento*, pero seleccionando únicamente los valores negativo o positivo, es decir, prescindiendo de aquellas filas donde el sentimiento sea neutro.

- Empleando la **Regresión Logística**, cree un *modelo de clasificación de texto* basándose en la variable **sentimiento**, pero seleccionando únicamente los valores **negativo** o **positivo**.
- Usando el algoritmo **Naive Bayes**, cree un *modelo de clasificación de texto* basándose en la variable **sentimiento**, pero seleccionando únicamente los valores **negativo** o **positivo**.

Pregunta 3. Grandes Modelos de Lenguaje (LLM)

El dataset **documentos** contiene 120 oraciones. Descripción de las columnas:

- **doc_id**: Identificador del documento.
- **texto**: El texto del documento.

Sabemos de antemano que cada uno de los documentos pertenece a una de estas ocho **categorías**: *Cine, Cocina, Deportes, Economía, Historia, Literatura, Naturaleza, Tecnología*. De cada categoría hay 15 documentos.

Se pide:

- **Clustering con BERT**

Empleando *Sentence Transformers*, utilizar el modelo `hiiamsid/sentence_similarity_spanish_es` para obtener los *embeddings* de los documentos.

Visualizar los embeddings mediante *UMAP*.

Realizar el *clustering*, asignando a cada documento una de las ocho categorías anteriormente indicadas.

- **Clustering con Nomic Bert**

Empleando *Ollama*, usar el modelo `nomic-embed-text-v2-moe` para obtener los *embeddings* de los documentos.

Visualizar los embeddings mediante *UMAP*.

Realizar el *clustering*, asignando a cada documento una de las ocho categorías anteriormente señaladas.

- **Clasificación de texto con Qwen3**

Empleando *Ollama*, usar el modelo `qwen3:4b` para clasificar los documentos, asignando a cada documento una de las ocho categorías anteriormente mencionadas.

Recomendaciones a seguir: No utilizar la salida estructurada, incluir en el *prompt* las ocho categorías.

- **Comparar la clasificación de Qwen3 con el clustering de BERT**

Mostrar aquellos documentos no coincidentes, es decir, la categoría asignada con *Qwen3* es distinta al grupo asignado con *BERT*.

- **Comparar la clasificación de Qwen3 con el clustering de Nomic Bert**

Mostrar aquellos documentos no coincidentes, es decir, la categoría asignada con *Qwen3* es distinta al grupo asignado con *Nomic Bert*.

Pregunta 4. Grandes Modelos de Lenguaje (LLM)

Queremos extraer cierta información de varias páginas web de *Wikipedia* (URL), referentes a 10 personajes ilustres: "Albert Camus", "Marie Curie", "Albert Einstein", "Alexander Fleming", "Mahatma Gandhi", "Gabriel García Márquez", "Nelson Mandela", "Severo Ochoa", "Max Planck" y "Santiago Ramón y Cajal".

Páginas web (URL) a procesar:

```
urls <- c(
  "https://es.wikipedia.org/wiki/Albert_Camus",
  "https://es.wikipedia.org/wiki/Marie_Curie",
  "https://es.wikipedia.org/wiki/Albert_Einstein",
  "https://es.wikipedia.org/wiki/Alexander_Fleming",
  "https://es.wikipedia.org/wiki/Mahatma_Gandhi",
  "https://es.wikipedia.org/wiki/Gabriel_García_Márquez",
  "https://es.wikipedia.org/wiki/Nelson_Mandela",
  "https://es.wikipedia.org/wiki/Severo_Ochoa",
  "https://es.wikipedia.org/wiki/Max_Planck",
  "https://es.wikipedia.org/wiki/Santiago_Ramón_y_Cajal")
```

De cada personaje, queremos obtener la siguiente información:

- *Nombre*: Nombre de la persona.
- *Nacimiento*: Fecha y lugar de nacimiento.
- *Resumen*: Un resumen de su principal logro.
- *Nobel*: Disciplina en la que ganó el Premio Nobel.
- *Año*: Año en que ganó el Premio Nobel.
- *Fallecimiento*: Fecha y lugar de fallecimiento.

Se puede dar el caso, en el que algún personaje tenga más de un premio nobel (*Marie Curie*).

También, se puede dar el caso, en el que algún personaje nunca fue galardonado con el premio nobel (*Mahatma Gandhi*).

Se pide:

- **Gemini**

Utilizando el paquete `ellmer`:

- Definir la salida estructurada a utilizar en el modelo.
 - Emplear la función `google_tool_web_fetch` para extraer la información de cada página web.
 - Usar el modelo *Gemini* para obtener la información requerida.
- Es aconsejable hacer una pausa (30 o 45 segundos) entre peticiones, para no saturar el servidor.

- **Ollama**

- Utilizar la *API Web fetch* de Ollama para extraer el contenido de las páginas web indicadas.
- Definir la salida estructurada a utilizar en el modelo.
- Usar el modelo `qwen3:4b` para obtener la información requerida.